Dynamic Programming Study Booklet

Redesigned from handwritten classroom notes into a modern study guide

Cleaned, reorganized, repeated content merged, weak pages omitted.

Preface

Welcome to the **Dynamic Programming Study Booklet**. This guide is a carefully reconstructed and polished version of original handwritten classroom notes. The aim is to provide a clean, typed, and structured approach to mastering Dynamic Programming (DP) patterns.

Dynamic programming often feels like magic until you recognize the recurring patterns. Instead of memorizing hundreds of problems, this booklet focuses on the core families: 0/1 Knapsack, Subset Sum variations, Unbounded Knapsack, Longest Common Subsequence (LCS), Matrix Chain Multiplication (MCM), and Egg Dropping.

How this booklet is organized:

- **Chapters 1-7:** Core patterns with problem statements, recurrence relations, and examples.
- **Final Revision Toolkit:** A quick-reference formula sheet and pattern map.
- **Practice Problems:** A mixed set of questions to test your pattern recognition.
- **Model Solutions:** Concise outlines for all practice problems.

Table of Contents

Chapter 1: Dynamic Programming Foundations

Chapter 2: 0/1 Knapsack Pattern

Chapter 3: Partition, Subset Sum, and Difference

Chapter 4: Unbounded Knapsack Family

Chapter 5: Longest Common Subsequence Family

Chapter 6: Matrix Chain Multiplication

Chapter 7: Egg Dropping

Final Revision Toolkit

Practice Problems

Model Solutions

Dynamic Programming Foundations

Dynamic Programming is a method for solving complex problems by breaking them down into simpler overlapping subproblems. At its core, **DP = Recursion + Storage**.

When this pattern appears:

- The problem asks for an optimal value (maximum, minimum, longest, shortest).
- The problem asks for the total number of ways to do something.
- Future decisions depend on current choices (overlapping subproblems).

Memoization vs. Tabulation

Feature	Memoization (Top-Down)	Tabulation (Bottom-Up)
Approach	Recursive calls with a cache array/matrix.	Iterative loops filling a DP table.
Ease of Writing	Usually easier, directly translates from base recursion.	Requires understanding topological order of subproblems.
Overhead	Function call stack overhead.	No recursion overhead, generally faster constant time.

Exam Lens

Always write the pure recursive solution first. Once you verify it's correct, applying memoization requires only adding a lookup table. Tabulation is the final optimization step.

Key Takeaways

Identify the changing variables in your recursive functionâ€”these become the dimensions of your DP table.

0/1 Knapsack Pattern

Given weights and values of `n` items, put these items in a knapsack of capacity `W` to get the maximum total value. Each item can be picked at most once (0 or 1 time).

When this pattern appears:

- You have an array of items with associated constraints (weights) and profits (values).
- You have a total capacity limit.
- You must choose to include or exclude each item to maximize profit.

The Include/Exclude Choice

For each item `i`, you have two choices if its weight is less than or equal to the remaining capacity: include it or exclude it. If its weight is greater than the capacity, you must exclude it.

Core Recurrence

```
// Base Case / Initialization
for i from 0 to n: dp[i][0] = 0
for j from 0 to W: dp[0][j] = 0

// Recurrence
for i from 1 to n:
  for j from 1 to W:
    if wt[i-1] <= j:
      dp[i][j] = max(val[i-1] + dp[i-1][j - wt[i-1]], dp[i-1][j])
    else:
      dp[i][j] = dp[i-1][j]
```

Exam Lens

Remember that the DP table dimensions are `(n+1) x (W+1)`. The index `i` in the DP table corresponds to the item at index `i-1` in the original weights and values arrays.

Key Takeaways

The 0/1 Knapsack is the parent of all subset choice problems. Mastering this recurrence makes Chapter 3 trivial.

Partition, Subset Sum, and Difference Problems

This family of problems uses the exact same mechanics as 0/1 Knapsack, but applied to arrays of numbers where we usually care about the sum of subsets.

Subset Sum (Decision)

Given an array and a target sum, does there exist a subset that adds up to the target? We replace `max()` with logical `OR (||)`.

```
if arr[i-1] <= j:
  dp[i][j] = dp[i-1][j - arr[i-1]] || dp[i-1][j]
else:
  dp[i][j] = dp[i-1][j]
```

Equal Sum Partition

Can we divide the array into two subsets with equal sums?

- **Insight:** If the total sum is odd, it's impossible. Return false.
- If the total sum is even, the target sum for one subset is exactly `total_sum / 2`. Call Subset Sum with this target.

Minimum Subset Sum Difference

Divide an array into two subsets S1 and S2 to minimize `|S1 - S2|`.

Since S1 + S2 = sum, we want to minimize `|sum - 2*S1|`. We find all possible valid subset sums using the Subset Sum DP table (last row), and pick the valid S1 that minimizes the difference.

Count Subsets with Given Difference

Count how many ways to partition the array such that S1 - S2 = diff. We know S1 + S2 = sum. Therefore, `2*S1 = sum + diff`, so `S1 = (sum + diff) / 2`.

If `(sum + diff)` is odd, the answer is 0. Otherwise, find the number of subsets that sum to `(sum + diff) / 2` using addition in the recurrence instead of OR/max.

Exam Lens

For counting subsets, initialize `dp[i][0] = 1` (there is 1 way to make sum 0: empty subset). Use `+` instead of `max` or `||`.

Unbounded Knapsack Family

In the Unbounded Knapsack problem, you are allowed to use the same item an unlimited number of times.

When this pattern appears:

- You have an unlimited supply of items (e.g., coins of given denominations).
- You need to make a target amount exactly.

Core Recurrence

The only change from 0/1 Knapsack is that when we include an item, we do not move to the previous item (`i-1`). We stay on the same item (`i`) because we can pick it again.

```
if wt[i-1] <= j:
  dp[i][j] = max(val[i-1] + dp[i][j - wt[i-1]], dp[i-1][j]) //
Note: dp[i] not dp[i-1]
else:
  dp[i][j] = dp[i-1][j]
```

Variations

- **Rod Cutting:** Exactly unbounded knapsack. Lengths are weights, prices are values.
- **Coin Change (Ways):** Count the total number of ways to make a sum. Use addition.
- **Coin Change (Minimum Coins):** Find the minimum number of coins to make a sum. Use `min` and initialize the table with infinity.
- **Max Ribbon Cut:** Similar to rod cutting but asking for maximum pieces.

Longest Common Subsequence (LCS) Family

LCS problems involve comparing two strings or sequences to find common elements while maintaining relative order.

Core Recurrence (LCS)

If characters match, the length increases by 1. If not, we take the max of skipping a character in string X or string Y.

```
if X[i-1] == Y[j-1]:
  dp[i][j] = 1 + dp[i-1][j-1]
else:
  dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

Variations

- **Longest Common Substring:** If characters don't match, reset to 0 instead of taking max. The answer is the maximum value in the entire DP table.
- **Shortest Common Supersequence (SCS):** Length = `m + n - LCS(X, Y)`.
- **Print LCS:** Traverse the DP table backwards from `dp[m][n]`. If characters match, add to result and go diagonally. Else, move to the larger value (up or left).
- **Min Insertions/Deletions (X to Y):** Deletions = `length(X) - LCS`. Insertions = `length(Y) - LCS`.
- **Longest Palindromic Subsequence (LPS):** `LPS(s) = LCS(s, reverse(s))`.
- **Min Deletions to Palindrome:** `length(s) - LPS(s)`.
- **Sequence Pattern Matching:** Is string A a subsequence of string B? Check if `LCS(A, B) == length(A)`.

Key Takeaways

Almost all string comparison problems can be reduced to LCS. Finding palindromes? Reverse the string and run LCS!

Matrix Chain Multiplication

MCM problems involve placing parentheses in an expression or dividing an array into intervals to minimize or maximize a cost.

When this pattern appears:

- You need to evaluate an expression or sequence in different orders.
- The problem asks for minimum/maximum cost of merging or multiplying.

Core Recurrence

We pick a split point `k` between `i` and `j-1`, solve for the left half, solve for the right half, and add the cost of merging them.

```
solve(i, j):
  if i >= j: return 0
  min_cost = infinity
  for k from i to j-1:
    cost = solve(i, k) + solve(k+1, j) + arr[i-1] * arr[k] *
arr[j]
    min_cost = min(min_cost, cost)
  return min_cost
```

Exam Lens

Identifying the bounds for `i`, `j`, and `k` is the hardest part. Usually `i = 1` and `j = n-1`. `k` goes from `i` to `j-1`.

Egg Dropping

Given `e` eggs and `f` floors, find the minimum number of attempts needed in the worst-case to find the critical floor from which an egg breaks.

Core Recurrence

If we drop an egg from floor `k`, two things can happen:

1. **Egg breaks:** We have `e-1` eggs and must check the `k-1` floors below.
2. **Egg survives:** We still have `e` eggs and must check the `f-k` floors above.

We want the *worst-case* outcome of the drop, so we take the max of the two. We want to choose floor `k` to *minimize* this worst-case.

```
solve(e, f):
  if f == 0 or f == 1: return f
  if e == 1: return f

  min_attempts = infinity
  for k from 1 to f:
    attempts = 1 + max(solve(e-1, k-1), solve(e, f-k))
    min_attempts = min(min_attempts, attempts)

  return min_attempts
```

Optimization

Since `solve()` is called twice inside the loop, we can memoize both calls independently to greatly reduce time complexity.

Final Revision Toolkit

Problem Recognition Checklist

- **Array + Target Sum + Combinations?** â†’ Knapsack / Subset Sum
- **Unlimited supply?** â†’ Unbounded Knapsack
- **Two Strings + Match/Compare?** â†’ LCS
- **One String + Palindrome?** â†’ LPS (LCS with reverse)
- **Array + Partition into Intervals + Cost?** â†’ MCM
- **Minimizing worst-case scenarios?** â†’ Egg Dropping (Minimax)

Time Complexity Snapshot

Pattern	Time Complexity	Space Complexity (Tabulation)
0/1 Knapsack	O(n * W)	O(n * W)
Unbounded Knapsack	O(n * W)	O(n * W) or O(W) optimized
LCS	O(m * n)	O(m * n)
MCM	O(nÂ³)	O(nÂ²)
Egg Dropping	O(e * fÂ²)	O(e * f)

Practice Problems

1. Target Sum / Count of Sign Assignments

Given an array and a target, assign + or - to each element so that the final expression equals the target. Count the number of valid assignments.

2. Minimum Number of Coins

Given coin denominations and a total amount, find the minimum number of coins needed to make that amount.

3. 0/1 Knapsack Value

For weights {1, 3, 4, 5}, values {1, 4, 5, 7}, and capacity 7, compute the maximum profit.

4. Partition Equal Subset Sum

Determine whether the array {1, 5, 11, 5} can be partitioned into two subsets with equal sum.

5. Minimum Subset Sum Difference

For the array {1, 6, 11, 5}, find the minimum possible difference between two subset sums.

6. Rod Cutting

Given rod length 4 and prices {1, 5, 8, 9}, find the maximum obtainable revenue.

7. Longest Palindromic Subsequence

Given the string `agbcba`, find the longest palindromic subsequence length.

8. Shortest Common Supersequence

Given strings `AGGTAB` and `GXTXAYB`, find the length of the shortest common supersequence.

9. Matrix Chain Multiplication

For dimensions {40, 20, 30, 10, 30}, find the minimum number of scalar multiplications.

10. Egg Dropping Puzzle

Find the minimum number of trials needed in the worst case for 2 eggs and 10 floors.

Model Solutions

> **1. Target Sum / Count of Sign Assignments:** Convert the sign problem into a subset-difference problem. If the array sum is `sum` and target is `diff`, then count subsets with target `(sum + diff)/2`. If `sum + diff` is odd, the answer is 0.

> **2. Minimum Number of Coins:** This is an unbounded knapsack variant. Use `dp[i][j] = min(1 + dp[i][j-coin[i-1]], dp[i-1][j])`, with unreachable states initialized to a large value.

> **3. 0/1 Knapsack Value:** Try include/exclude decisions or fill the DP table. The best valid pick is weight 3 + 4 = 7 with value 4 + 5 = 9, so the maximum profit is **9**.

> **4. Partition Equal Subset Sum:** The total sum is 22, which is even. So check subset sum for target 11. Since {11} or {5,5,1} works, the answer is **true**.

> **5. Minimum Subset Sum Difference:** One optimal partition is {1, 5, 6} and {11}. Their sums are 12 and 11, so the minimum difference is **1**.

> **6. Rod Cutting:** Treat each cut length as reusable. For length 4, the best revenue is achieved by using a length-2 cut twice: 5 + 5 = **10**.

> **7. Longest Palindromic Subsequence:** Compute `LCS(s, reverse(s))`. For `agbcba`, the LPS is `abcba`, so the length is **5**.

> **8. Shortest Common Supersequence:** First find `LCS(AGGTAB, GXTXAYB) = 4` (GTAB). Therefore SCS length = 6 + 7 - 4 = **9**.

> **9. Matrix Chain Multiplication:** Apply the split recurrence across all valid `k` values. For {40, 20, 30, 10, 30}, the minimum multiplication cost is **26000**.

> **10. Egg Dropping Puzzle:** Use the worst-case recurrence with memoization. For 2 eggs and 10 floors, the minimum required trials in the worst case are **4**.

Final Advice

Dynamic programming is about recognizing the structure. Focus on the core patterns. If you see two strings, immediately think LCS DP table. If you see array subsets, think Knapsack. Draw the DP grid, write the recurrence on paper, and the code will follow.

Good luck with your studies!